

Proposta de Software

1. Identificação

Nome do projeto: VoltQuest

Nome do(a) aluno(a): Victória Lucas Chaves

Repositório GitHub: <https://github.com/VictorialChaves/VoltQuest.git>

Obs.: O professor ([Clique33](#)) deve ser adicionado como colaborador com acesso total.

2. O Projeto

Descrição: (Breve descrição do software a ser desenvolvido, destacando o problema e o público-alvo.)

O projeto consiste no desenvolvimento de um jogo digital educativo focado no ensino intuitivo de eletrônica básica e fundamentos de robótica. A aplicação aborda o problema da excessiva abstração teórica e da falta de laboratórios práticos no ensino de ciências e tecnologia, que frequentemente geram desinteresse por áreas de engenharia. Por meio de uma dinâmica gamificada de resolução de problemas no desenvolvimento de projetos e simulação visual de circuitos, o jogador aprende a dimensionar e conectar componentes reais (como sensores, atuadores e fontes) de forma prática. O público-alvo abrange estudantes da educação básica (Ensino Fundamental e Médio).

Objetivo: (Descreva o propósito do sistema e os benefícios esperados.)

Projetar e desenvolver um jogo digital educativo com simulação interativa de circuitos e robótica, promovendo a aprendizagem ativa e prática por meio de desafios gamificados. Como benefícios, espera-se desmistificar o ensino de eletrônica e programação para iniciantes de diversas idades, aumentar a retenção de conceitos fundamentais (como polaridade, sensoriamento e controle) através da experimentação sem risco de danos

materiais, além de estimular o raciocínio lógico e o interesse por carreiras nas áreas de engenharia e tecnologia.

Escopo: (Liste as principais funcionalidades planejadas.)

1. Oficina de Montagem Modular Passo a Passo: Interface guiada que orienta o usuário no posicionamento de cada peça mecânica e componente eletrônico em projetos temáticos, como um carrinho seguidor de linha, braço robótico ou sistema de automação residencial.
2. Enciclopédia Interativa de Componentes: Janela explicativa detalhando a função, polaridade, símbolos e aplicações práticas de cada elemento selecionado (ex.: resistores, sensores LDR, pontes H, servomotores e microcontroladores).
3. Editor e Simulador de Ligações Elétricas: Sistema visual para conectar fios entre terminais de componentes em uma bancada/protoboard virtual, com validação de nós de circuito em tempo real.
4. Simulação de Falhas: Respostas visuais e sonoras dinâmicas a erros de projeto, demonstrando queima de componentes por sobretensão/sobrecorrente, curtos-circuitos ou conexões com polaridade invertida.
5. Testes Dinâmicos de Funcionamento: Ambiente interativo onde o jogador aciona o robô montado para testar sua resposta a estímulos externos (ex.: carrinho seguindo trajetos ou braço robótico manipulando objetos).

3. Tecnologias e Ferramentas (Coloque somente categorias que forem relevantes ao seu projeto.)

Categoria	Ferramenta/Tecnologia	Justificativa
Linguagem de programação	GDScript (e/ou C#)	Linguagem de programação do Godot Engine
Framework/Biblioteca	Godot Engine	Criação de jogos de código aberto, gratuito e leve, ideal para a criação de sistemas modulares em 2D baseados em nós e cenas.
Banco de dados	Arquivos JSON	Como o foco do projeto é a execução local e

		acessibilidade rápida via Web sem exigir infraestrutura pesada de servidores, o armazenamento via JSON é a solução mais eficiente.
Prototipagem e Design de Interface (UI/UX)	Figma	Utilizado para planejar o fluxo do usuário, projetar o layout visual das bancadas de montagem, janelas de diálogo e arquitetar a identidade visual da Enciclopédia de Componentes antes da implementação no motor de jogos.
Criação de Arte e Sprites 2D	Aseprite / Inkscape	Ferramentas essenciais para o desenho e vetorização gráfica dos componentes eletrônicos (resistores, protoboards, LEDs, sensores LDR, servomotores) e animações visuais de falhas (como queima e faíscas).

4. Backlog do Produto (Apenas os épicos principais do projeto. Abaixo um exemplo.)

ID	História do Usuário
#01	Como usuário, quero criar uma conta para acessar o sistema.
#02	Como usuário, quero acessar meu perfil no sistema
#03	Como usuário, quero selecionar projetos temáticos em um menu de fases, para que eu possa escolher o nível de complexidade desejado para praticar.

- #04 Como usuário, quero clicar em qualquer equipamento da bancada para visualizar sua simbologia e função, para que eu compreenda para que serve cada componente antes de usá-lo no circuito.
- #05 Como usuário, quero receber orientações visuais sobre onde inserir cada peça mecânica e elétrica na estrutura do projeto, para que eu consiga montar o robô de forma ordenada e correta.
- #06 Como usuário, quero arrastar fios e interligar os terminais dos componentes na bancada/protoboard virtual, para que eu possa fechar os circuitos elétricos do meu projeto
- #07 Como usuário, quero visualizar respostas visuais imediatas (como fumaça, faíscas ou alertas) caso inverta a polaridade ou cause sobretensão, para que eu aprenda com o erro sem riscos físicos
- #08 Como usuário, quero colocar o robô finalizado em uma pista ou bancada de teste interativa 2D, para que eu possa verificar se ele responde corretamente aos sensores e comandos planejados
- #09 Como usuário, quero salvar o estado do projeto e poder voltar à tela inicial a qualquer momento sem perder as alterações feitas, para que eu possa continuar a montagem posteriormente de onde parei
- #10 Como usuário, quero acessar uma oficina livre com todos os componentes desbloqueados, para que eu possa experimentar combinações personalizadas sem a obrigação de seguir metas de fases.

Game Design Document (GDD)

1. Visão Geral (High Concept)

- Gênero: Quebra-cabeça / Simulação Educativa / Engenharia.
- Plataforma: PC / Web (Navegador).
- Pitch em uma frase: Um jogo de simulação e montagem onde o jogador constrói circuitos e robôs reais em uma bancada virtual, vendo suas criações ganharem vida (ou pegarem fogo com segurança!) em desafios práticos.
- Referências de Mercado: Tinkercad Circuits + Assemble with Care + eletrician simulator (simplificado para educação).

2. Pilares de Design (Design Pillars)

Pilar	Descrição e Experiência do Jogador
1. Resolução Prática de Problemas (Problem Solving)	O prazer vem de entender porque um circuito não funciona, ajustar um resistor ou mudar uma fiação e ver o robô finalmente completar o objetivo.
2. Falha Segura e Divertida (Humorous/Low-Stakes Failure)	Errar não pune o jogador com telas chatas de "Game Over", mas com feedbacks visuais dinâmicos (fumaça saindo do LED, fusível estourando) que ensinam o erro sem frustrar.
3. Criatividade Tangível (Maker Mastery)	Fazer o jogador sentir que está mexendo com ferramentas de verdade (protoboard, alicate, fios coloridos, motores) e que tem autonomia para montar e testar.

3. Público-Alvo e Limitações de Escopo

- Quem é o jogador: Estudantes do Ensino Fundamental II e Médio (11 a 17 anos), professores de ciências/robótica e iniciantes em eletrônica.

- Para quem o jogo NÃO é: Engenheiros experientes em busca de simulação industrial. O foco é intuição e fundamentos, não cálculo diferencial de transitórios.
- Tom e Estilo: Educativo, encorajador, prático e levemente bem-humorado nas animações de falha.

4. Loop Principal de Gameplay (Core Game Loop)

[Desafio / Pedido]



[Escolha de Peças e Consulta à Enciclopédia]



[Conexão na Bancada / Protoboard]



[Ligar Energia / Simulação]

└─ Erro (Curto/Queima) ──→ Análise da Falha e Ajuste

└─ Sucesso ───────────→ [Pista de Teste Dinâmica] ──→ Desbloquear Novo Projeto

5. Mecânicas Principais (Escopo do MVP / Protótipo)

A. Bancada de Montagem e Editor de Fios

- Navegação: Interface em visão isométrica ou topo (2D)
- Fiação: Ponto a ponto (arrastar e soltar entre furos da protoboard e terminais).
- Código de Cores: Fios automáticos ou customizáveis (Vermelho = VCC, Preto = GND, etc.).

B. Sistema de Falhas e Validação em Tempo Real

- Verificações Básicas:
 - Polaridade invertida (LED não acende ou queima se não houver diodo de proteção).
 - Sobretensão/Corrente excessiva (LED queima sem resistor limitador).
 - Curto-circuito direto na fonte (bateria esquenta / fusível desarma).

- Feedback: Animação 2D/3D de fumacinha cinza, estalo sonoro cômico e indicação clara no componente: "Queimado por excesso de corrente (falta de resistor)".

C. Enciclopédia Interativa (Tooltip & Card)

- Clicar com botão direito em qualquer peça abre um pop-up:
 - O que faz: Explicação em 2 linhas.
 - Cuidado: Ex: "Tem lado certo (+ e -)".
 - Símbolo esquemático: Mostra como ele aparece em esquemas reais.

D. Modo de Teste Dinâmico (Sandbox de Execução)

- Botão "Testar Robô": a câmera transiciona da bancada para a pista de teste:
 - Exemplo: Carrinho tenta seguir a linha preta com base nos LDRs montados; se a lógica ou circuito estiver errada, ele bate na parede ou não sai do lugar.

6. Conteúdo Inicial para o Protótipo (Primeiras 4 Fases/ cenários diferentes do MVP)

Fase	Projeto	Conceito Ensinado	Critério de Sucesso
01	Primeiros Passos do Robô - Acionamento Motor DC Básico	Circuito elétrico fechado, polaridade contínua (DC) e conversão de energia elétrica em movimento mecânico.	Encaixar cada elemento e conectar a bateria aos motores DC através de uma chave liga/desliga, garantindo que as rodas girem para frente sem curto-circuito.
02	Lanterna Simples	Circuito fechado, chave liga/desliga e resistor limitador de LED.	Ligar o LED sem queimá-lo ao acionar o interruptor.
03	Poste Automático	Divisor de tensão com sensor de luz (LDR).	O LED deve acender apenas quando o jogador apaga a luz ambiente.

04	Carrinho Seguidor Básico	Motores DC acionados por sensores ópticos/LDR.	O carrinho deve percorrer uma pista curva simples do início ao fim.
----	--------------------------	--	---

7. Direção de Arte e Áudio

- Estilo Visual: Cores contrastantes e vibrantes, estilo flat/vetorial limpo ou 3D estilizado (low-poly)/ ou vintage.
- Legibilidade: Terminais e furos de conexão devem ter destaque visual ao passar o mouse (hover highlight).
- Áudio: Efeitos sonoros táteis e satisfatórios (cliques de encaixe, estalo de interruptores, som de motor acelerando, "pop" suave de curto-circuito).

Segue abaixo dois prints do jogo Assemble with Care, para inspiração de bancada.

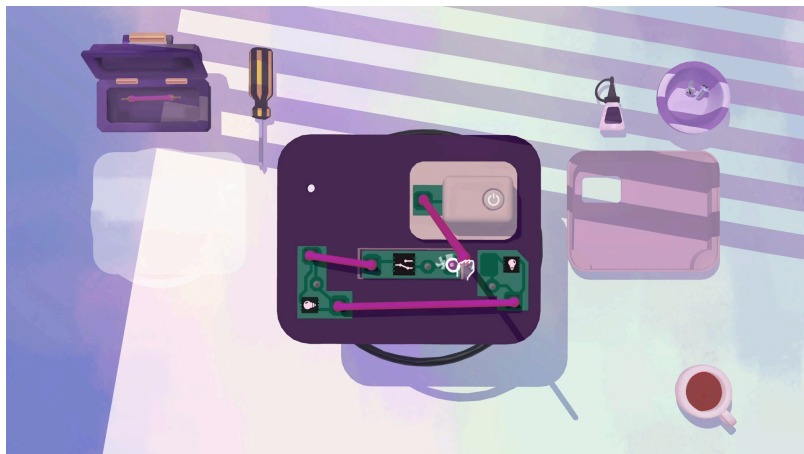
Figura 1 - exemplo de Design do jogo Assemble with Care



Fonte:

https://store.steampowered.com/app/1202900/Assemble_with_Care/?l=brazilian&curator_clanid=41107206

Figura 2 - exemplo de Design do jogo Assemble with Care



Fonte: <https://cthulhuscritiques.com/2021/03/28/assemble-with-care-review/>

8. Modelo de Distribuição

- Fase 1 (MVP/Acadêmico): Executável Web gratuito para testes em sala de aula e coleta de dados de retenção e usabilidade.

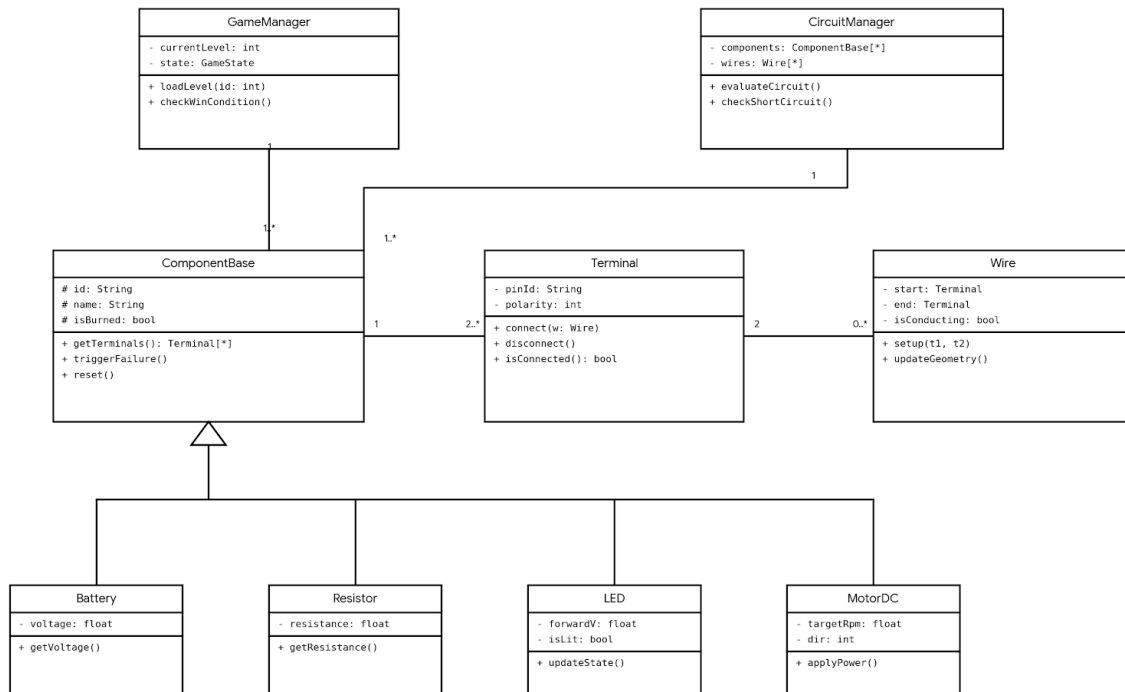
Modelo: Diagrama de classes

Desenvolvida com o suporte do Gemini, segue em anexo o diagrama de classes do jogo VoltQuest.

Figura 3 - Diagrama de classes

Diagrama de Classes de Domínio - VoltQuest

Padrão UML (Engenharia de Software Moderna)



1. Os Controladores Globais (*Singletons*)

GameManager (O Diretor do Jogo)

- O que faz: Controla as regras de videogame (pontuação, fases, fluxo de telas).
- Atributos:
 - - **currentLevel: int**: Guarda o ID da fase atual (ex: 1 para "Acender LED", 4 para "Seguidor de linha").
 - - **state: GameState**: Diz o que o jogador está fazendo agora (ex: *Editando, Rodando Teste, Falhou, Venceu*).
- Métodos:
 - + **loadLevel(id)**: Limpa a tela e carrega as peças e objetivos da próxima fase.
 - + **checkWinCondition()**: É chamado toda vez que o circuito muda. Verifica se o robô chegou no final da pista ou se a luz acendeu, declarando a vitória.

CircuitManager (O Motor Físico/Elétrico)

- O que faz: É o coração da simulação. Ele olha para as peças na tela como se fossem um Grafo (matemática discreta), onde os **Terminais** são os nós e os **Wires** (Fios) são as arestas.
- Atributos:
 - - **components: ComponentBase[*]**: Uma lista (array) contendo todas as peças que estão na bancada naquele momento.
 - - **wires: Wire[*]**: Uma lista de todos os fios desenhados.
- Métodos:
 - + **evaluateCircuit()**: Varre a malha. Calcula as resistências equivalentes e correntes com base nas Leis de Kirchhoff. (laço de repetição (**while** ou **for**) que percorra as conexões (fio -> pino -> componente -> pino -> fio) e some os valores.)
 - + **checkShortCircuit()**: Varredura de segurança. Se a corrente tender ao infinito (bateria ligada direto no GND sem resistência), ele detecta e manda o aviso de queima.

2. A Estrutura Física (O Circuito)

Representam o mundo físico da bancada.

ComponentBase (A Classe Mãe)

- O que faz: É um modelo genérico. Você nunca cria um "ComponentBase" no jogo; você cria LEDs, motores, etc. Mas todos eles herdam essas características básicas para que o **CircuitManager** possa tratar todos de forma padronizada (Polimorfismo).
- Atributos:
 - **# id, # name**: Identificadores únicos da peça.
 - **# isBurned: bool**: Sinaliza se a peça estourou por erro do jogador.
- Métodos:
 - + **getTerminals()**: Devolve a lista de pinos metálicos daquela peça.
 - + **triggerFailure()**: Toca o som de explosão, solta fumaça na tela e muda a imagem da peça para torrada.

- `+ reset()`: Restaura o componente para seu estado original

Terminal (Os Pinos de Conexão)

- O que faz: Representa os furos da protoboard, as pernas do LED ou os bornes do motor.
- Atributos:
 - `- pinId: String`: Ex: "Anodo", "Catodo", "VCC", "GND".
 - `- polarity: int`: Pode restringir o pino (ex: um pino que tem que ser positivo).
- Métodos:
 - `+ connect(Wire) / + disconnect()`: Associa ou desassocia um fio ao pino.
 - `+ isConnected()`: Responde se tem algo ligado ali.

Wire (O Fio Condutor)

O que faz: A ponte visual e lógica entre dois Terminais.

Atributos:

- `- start, - end`: Terminais de origem e destino do fio.
- `- isConducting: bool`: Define o estado visual (brilhante/verde se conduzindo corrente, opaco se aberto).

Métodos:

- `+ setup(t1, t2)`: Crava o fio entre dois pinos ao soltar o clique do mouse.
- `+ updateGeometry()`: Atualiza os vetores da linha visual caso o jogador mova o componente pela mesa.

3. As Especializações (Herança)

As classes na parte de baixo do diagrama são as peças reais. Elas já têm tudo que o `ComponentBase` tem, mas adicionam dados específicos de engenharia elétrica:

- Battery/ Bateria (Fonte de Tensão):
 - Tem o atributo de tensão (`voltage`).
 - O método `getVoltage()` é consumido pelo `CircuitManager` para iniciar o cálculo da malha.

- Resistor (Carga Passiva):
 - Guarda a resistência em Ohms (**resistance**).
- LED (Diodo Emissor):
 - Possui a tensão de queda direta (**forwardV**) e um estado visual (**isLit**).
 - Seu método **updateState()** é chamado pelo simulador: se a corrente recebida for suficiente, ele acende a luz na tela. Se for excessiva, ele chama o **triggerFailure()** que herdou da mãe.
- MotorDC (Atuador Mecânico):
 - Traduz a elétrica para a robótica. Guarda a velocidade alvo (**targetRpm**) e a direção da rotação (**dir** = 1 para frente, -1 para trás).
 - **applyPower()**: Faz a roda do carrinho começar a girar na tela de testes dinâmicos (converte a energia para que as rodas girem na pista de testes).